

Programming Blue3: An SMT solver for Caprice-Lang

By: Nathanael Oh

Blue3 is an SMT solver for JHU's `caprice-lang`. It is used by the typechecker's Concolic Evaluator, or `ceval`.

Before Blue3, `ceval` used Z3 directly. Z3 is powerful, but often overkill for simple formulas like:

$$(6 \leq a) \wedge (a < 0)$$

This formula is **UNSAT**, because a cannot be at least 6 and less than 0 at the same time. Since calling Z3 has overhead, Blue3 was built to solve simple cases internally and fall back to Z3 when needed.

I will reference both theory bits along with implementation details throughout the report, leaning a bit more on the implementation side since this paper is meant for a general computer science audience. The full source code can be found on the `caprice-lang` repository if want to see it for yourself.

Intro

Blue3 is a small SMT solver with a full SAT/SMT pipeline. In benchmarks, Blue3's frontend was just over 60% faster than Z3 on simple formulas.

avg_blue3	avg_z3
222.0 μ s	329.0 μ s

When Blue3 cannot solve a formula, it falls back to Z3. This adds about 20.24 μ s of overhead on average, or roughly 5%.

num_slow_cases	avg_slower_by	avg_percent_slower
38	20.24 μ s	4.59%

So Blue3 gives a good tradeoff: faster simple cases, with Z3 as a backup.

P = NP and Boolean Satisfiability

Oversimplifying, $P = NP$ asks:

If we can check a solution quickly, can we also find that solution quickly?

Problems solvable in polynomial time are in P . Problems whose solutions can be verified in polynomial time are in NP .

So another way to state $P = NP$ is:

If a solution can be verified efficiently, can it also be found efficiently?

We do not know. Most computer scientists believe $P \neq NP$, but no one has proven it.

This matters because some problems are **NP-complete**: every problem in NP can be reduced to them in polynomial time. If we found a polynomial-time algorithm for one NP-complete problem, every problem in NP could be solved efficiently.

3SAT and Boolean Satisfiability

3SAT asks:

Given a propositional formula in CNF with at most 3 literals per clause, is there some assignment that makes it true?

For example:

$$(p \vee q) \wedge (\neg p \vee \neg q)$$

is satisfiable because $p = \text{true}$ and $q = \text{false}$ makes the formula true.

But:

$$(p \vee q) \wedge (\neg p \vee q) \wedge (p \vee \neg q) \wedge (\neg p \vee \neg q)$$

is unsatisfiable because no assignment of p and q can satisfy every clause.

3SAT is important because it is NP-complete. If we could solve 3SAT in polynomial time, we would prove $P = NP$.

Blue3 obviously does not solve $P = NP$. Instead, it uses practical SAT/SMT techniques to solve many real formulas efficiently.

SMT extends SAT with theory constraints. For example:

$$(6 \leq a) \wedge (a < 0)$$

can be abstracted into:

$$p \wedge q$$

where p represents $(6 \leq a)$ and q represents $(a < 0)$. The SAT solver handles the boolean structure, while the theory solver checks whether the constraints are actually consistent.

Useful Terminology

A **formula** is a boolean-valued expression. In this report, “formula” usually means a formula in **CNF**.

A formula in **CNF** is an AND of clauses:

$$(p \vee q \vee r) \wedge (s \vee t \vee \neg u)$$

A **clause** is one OR-group. A **literal** is an atom with a sign, like p or $\neg p$. An **atom** is the unsigned condition underneath a literal.

We will use three main formula categories:

1. A **SAT formula** is purely propositional.
2. A **theory formula** is handled by a theory solver.
3. An **SMT formula** combines propositional logic with theory constraints.

A **solver** takes a formula and returns either **SAT**, usually with a satisfying model, or **UNSAT**.

Difference Logic

Blue3 is meant for formulas too simple to justify calling Z3, such as:

$$(6 \leq a) \wedge (a < 0)$$

In our benchmarks, many of these formulas were integer-heavy:

formula_id	formula
9	$(0 < a) \wedge ((a + 1) \leq a)$
8	$(0 < a) \wedge ((a + 1) \leq 1)$
56	$(\text{not } (a = 0)) \wedge ((a + 10) = 0)$
11	$(1 < a) \wedge (a < 0)$
88	$(0 < a) \wedge (a < 1)$

This led us to **Integer Difference Logic**, or **IDL**, a fragment of linear integer arithmetic where constraints compare differences between integer terms.

IDL literals have the shape:

$$x - y \leq c$$

where x and y are integer variables or 0, and c is an integer constant.

For example:

$$(6 \leq a) \wedge (a < 0)$$

can be rewritten as:

$$((0 - a) \leq -6) \wedge ((a - 0) \leq -1)$$

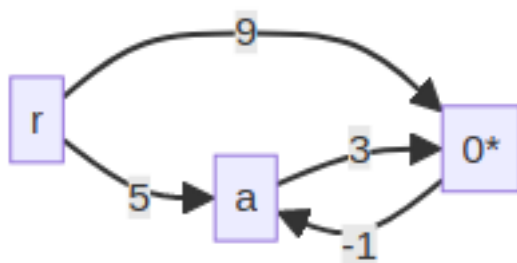
IDL gives Blue3 a mechanical way to recognize contradictions like this.

Bellman-Ford

Bellman-Ford finds shortest paths in a directed weighted graph. It also detects **negative cycles**, which are cycles whose total weight is negative.

For example:

```
let simple_no_neg =
  [ ("a", "0*", 3); ("0*", "a", -1)
  ; ("r", "0*", 9); ("r", "a", 5) ]
```



Running Bellman-Ford from r gives:

Node	Distance
a	5
0^*	8

The shortest path to 0^* is $r \rightarrow a \rightarrow 0^*$, with cost $5 + 3 = 8$. The cycle between a and 0^* has cost:

$$3 + (-1) = 2$$

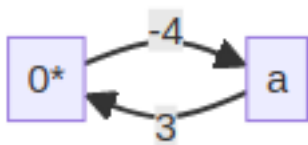
Since the cycle is positive, looping only makes paths more expensive.

Now change the edge $0^* \rightarrow a$ from -1 to -4 . The cycle cost becomes:

$$3 + (-4) = -1$$

Each loop makes the path cheaper, so Bellman-Ford reports a negative cycle.

```
let simple_neg =
  [ ("a", "0*", 3); ("0*", "a", -4)
  ; ("r", "0*", 9); ("r", "a", 5) ]
```



Relaxing Edges

Bellman-Ford repeatedly tries to improve distances. This is called **relaxation**.

An edge is relaxed when:

$$dist[from] + cost < dist[to]$$

In our implementation, we opt for `options` over explicit int-max values to represent the initial distances states; that first case where we find there is a min distance of `None` to the `to_` node represents the initial state.

So in either the initial state or any state after, we are effectively checking for $dist[from] + cost < dist[to]$:

```
utils/bellman_ford.ml
1 let relax_edge (tbl : tbl) (was_updated : bool) ((from_, to_, cost) as edge : Node.t edge) : bool =
2   match Hashtbl.find tbl from_, Hashtbl.find tbl to_ with
3   | (Some du, _), (None, _) ->
4     set_distance to_ tbl ~min:(du + cost) ~pred:edge
5   | (Some du, _), (Some dv, _) when du + cost < dv ->
6     set_distance to_ tbl ~min:(du + cost) ~pred:edge
7   | _ -> was_updated
```

The source starts at distance 0; every other node starts at infinity. Bellman-Ford relaxes edges at most $N - 1$ times, where N is the number of nodes, so we stop when we are at iteration number $N - 1$, or when a full pass does not update anything as a small optimization:

```
utils/bellman_ford.ml
```

```

1 let relax_edges (edges : Node.t edge list) (tbl : tbl) (i : int)
2   : [ `Continue of tbl | `Stop of tbl ] =
3   if i >= (Hashtbl.length tbl) - 1 then `Stop tbl
4   else
5     let is_dist_updated = List.fold_left (relax_edge tbl) false edges in
6     if is_dist_updated then `Continue tbl
7     else `Stop tbl

```

Predecessors and Negative Cycles

Each table entry stores:

(distance, predecessor edge)

The distance gives the shortest-known cost from the source. The predecessor edge lets us reconstruct the path.

After the normal relaxation loop, Bellman-Ford runs one extra pass. If any edge can still be relaxed, the graph has a negative cycle.

We invoke this extra pass via `find_relaxed_node_opt`:

utils/bellman_ford.ml

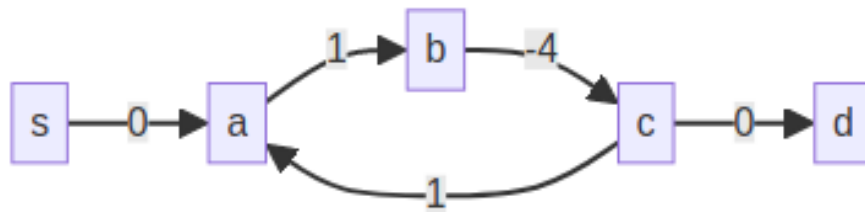
```

1 let find_relaxed_node_opt (edges : Node.t edge list) (dist : tbl) : Node.t option =
2   List.find_map (fun (_, to_, _) as edge) ->
3     if relax_edge dist false edge then
4       Some to_
5     else None)
6   edges

```

Where we just return the `to_` node of the first edge that was relaxed.

The relaxed node may not itself be inside the cycle:



So we have to walk backward through the predecessor links:

utils/bellman_ford.ml

```

1 let find_cycle_entry_opt (edges : Node.t edge list) (dist : tbl)
2   : Node.t option =
3   let num_nodes = Hashtbl.length dist in

```

```

4   let relaxed_predecessor = find_relaxed_node_opt edges dist in
5   match relaxed_predecessor with
6   | None -> None
7   | Some entry ->
8     let rec move_back node n =

```

By the pigeonhole principle, we need to backtrack a max of N nodes to guarantee we land inside the cycle:

Then we collect predecessor edges until we backtrack N times:

```

utils/bellman_ford.ml
1 let rec move_back node n =
2   if n = 0 then node
3   else if ...

```

Or until we loop back to the start:

```

utils/bellman_ford.ml
1 let rec move_back node n =
2   if n = 0 then node
3   else if n < num_nodes && node = entry then node
4   ...

```

Otherwise we continue backtracking:

```

utils/bellman_ford.ml
1 let rec move_back node n =
2   if n = 0 then node
3   else if n < num_nodes && node = entry then node
4   else
5     match find_predecessor node dist with
6     | None -> node
7     | Some from_ -> move_back from_ (n - 1)

```

In short:

1. Compute distances.
2. Run one extra relaxation pass.
3. If nothing changes, return distances.
4. If something changes, backtrack predecessors and return the negative cycle.

Bellman-Ford as a Difference Logic Solver

Bellman-Ford solves difference logic because each constraint can be encoded as a graph edge.

We encode:

$$x - y \leq c$$

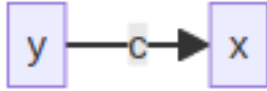
as:

$$(y, x, c)$$

It looks like this for Blue3:

```
smt/idl.ml
1 let leq_to_diff (left : Ints.affine) (right : Ints.affine) : diff =
2   match left, right with
3   | Var_plus_const (x, kx), Var_plus_const (y, ky) ->
4     { x = Node.symbol_key x ; y = Node.symbol_key y ; c = ky - kx }
5   ...
```

Where we pattern match to build our diff record.



Then we add a dummy source node with 0-weight edges to every node and run Bellman-Ford. If it finds a negative cycle, the formula is **UNSAT**. Otherwise, it is **SAT**.

Case 0. Split

IDL can solve inequalities like $\leq$, but disequality needs a split:

$$x \neq y \implies (x \leq y - 1) \vee (y + 1 \leq x)$$

```
smt/idl.ml
1 let find_split_opt (lit : 'k Theory.literal)
2   : 'k split_neq_case option =
3   let one = Formula.const_int 1 in
4   match lit with
5   | Neg Predicate (Equal, x, y) ->
6     ...
```

Where on that case, we return both lower and upper bounds:

```
smt/idl.ml
1 match lit with
2 | Neg Predicate (Equal, x, y) ->
3   begin match Ints.reflect_opt x, Ints.reflect_opt y with
```



```

4 | Some x', Some y' ->
5   let lower =
6     Theory.Predicate (Less_than_eq, x', Formula.minus y' one)
7   in
8   let upper =
9     Theory.Predicate (Less_than_eq, Formula.plus y' one, x')
10  in ...

```

We also include the equality case...

```

smt/idl.ml
1 begin match Ints.reflect_opt x, Ints.reflect_opt y with
2 | Some x', Some y' ->
3   ...
4   let eq = Theory.Predicate (Equal, x, y) in
5   Some (~lower:(Pos lower), ~upper:(Pos upper), ~eq:(Pos eq))

```

...because the SAT solver may still need to explore $x = y$ depending on context.

Case 1. SAT

Consider:

$$(-a \leq 1) \wedge (a \leq 2)$$

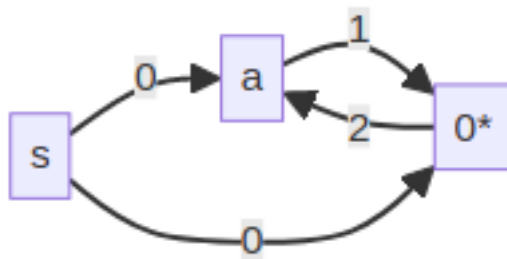
Encoding each literal gives:

$$-a \leq 1 \equiv 0^* - a \leq 1 \implies (a, 0^*, 1)$$

$$a \leq 2 \equiv a - 0^* \leq 2 \implies (0^*, a, 2)$$

After adding dummy source edges:

$$\text{Edges} = (a, 0^*, 1), (0^*, a, 2), (s, a, 0), (s, 0^*, 0)$$



Bellman-Ford finds no negative cycle, so the formula is **SAT**.

To get a concrete model, we normalize distances around the special 0* node:

$$\text{model}[x] = \text{distance}[x] - \text{distance}[0^*]$$

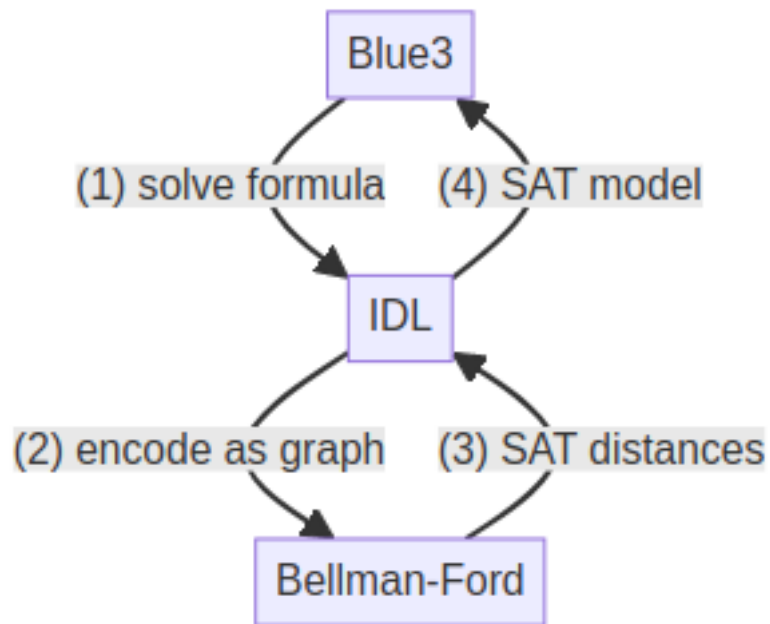
So all we need to do is get the the 0* node's distance `z0_dist`:

```
smt/idl.ml
1 let solve_int_diff (literals : 'k Theory.literal list)
2   : 'k Theory.theory_solution =
3   ...
4   let { edges ; edge_sources ; vars } = encode_graph lits in
5   match bellman_ford ~src:Node.root edges with
6   | `Negative_cycle edges -> ...
7   | `No_negative_cycle distances ->
8     let distance_map = NodeMap.of_list distances in
9     let z0_dist = NodeMap.find Node.zero distance_map in
```

And then subtract it from every other value with `var_dist - z0_dist`:

```
smt/idl.ml
1 ...
2 | `No_negative_cycle distances ->
3   let distance_map = NodeMap.of_list distances in
4   let z0_dist = NodeMap.find Node.zero distance_map in
5   let local_model =
6     vars
7     |> List.map (fun uid ->
8       let var_dist = NodeMap.find (Node.symbol_key uid) distance_map in
9       uid, Model.Int (var_dist - z0_dist))
10    |> Uid.Map.of_list
11    in
12    let model = Model.from_value_map local_model in
13    Theory.sat model
```

So the SAT flow looks like this:

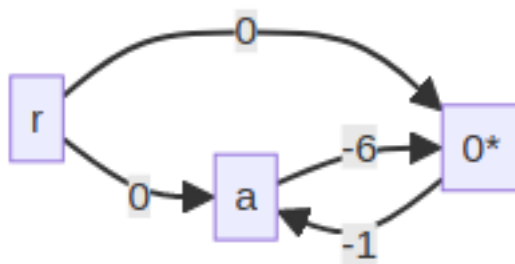


Case 2. UNSAT

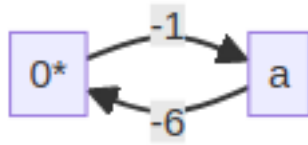
Now consider:

$$(6 \leq a) \wedge (a < 0)$$

This encodes to:



Bellman-Ford finds this negative cycle:



And returns the edges of the cycle:

```

smt/idl.ml
1 let solve_int_diff (literals : 'k Theory.literal list)
2   : 'k Theory.theory_solution =
3   ...
4   let { edges ; edge_sources ; vars } = encode_graph lits in
5   match bellman_ford ~src:Node.root edges with
6   | `Negative_cycle edges -> ...
  
```

The cycle edges map back to:

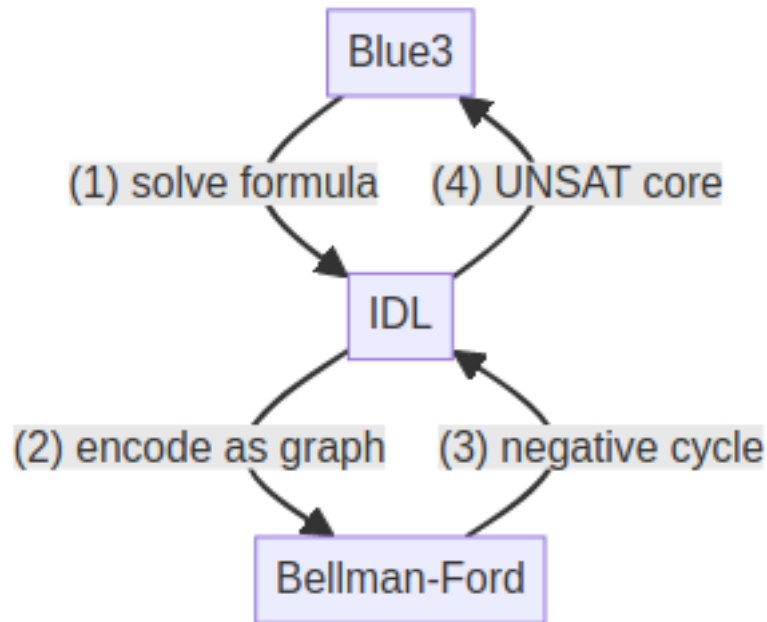
$$(a, 0^*, -6) \implies 6 \leq a$$

$$(0^*, a, -1) \implies a < 0$$

```

smt/idl.ml
1 match bellman_ford ~src:Node.root edges with
2 | `Negative_cycle edges ->
3   let core =
4     edges
5     |> List.filter_map (fun edge -> source_from_edge edge edge_sources)
6     |> List.sort_uniq compare
7   in
8   Theory.unsat core
  
```

So the original formula is **UNSAT**.



The negative cycle is also the **UNSAT core**, meaning the specific literals responsible for the contradiction.

SAT

3SAT was the first problem shown to be NP-complete. More generally, k-SAT asks:

Given a formula with at most k literals per clause, is it satisfiable?

For example:

$$(p \vee q \vee \neg r) \wedge (\neg p \vee r) \wedge (q \vee r) \wedge \neg r$$

is satisfiable with $p = \text{false}$, $q = \text{true}$, $r = \text{false}$.

No polynomial-time SAT algorithm is known, so practical SAT solvers are still very smart search algorithms.

Conflict-Driven Clause Learning

Blue3 uses **Conflict-Driven Clause Learning**, or CDCL, for boolean solving.

The loop is:

1. Infer forced assignments.

2. If nothing is forced, guess.
3. If there is a contradiction, learn from it.
4. Repeat until SAT or UNSAT.

Our `cdcl` function takes care of the main recursive loop by calling the “actual” `bcp` function:

```
sat/cdcl.ml
1 let cdcl formula = bcp 0 [] formula

sat/cdcl.ml
1 let rec bcp (level : int) (trail : Trail.trail) (formula : Formula.formula) : Solution.solution =
2   let model = Trail.to_model trail in
3   match unit_propagate formula model with
4   | Decide -> ...
5   | Conflict clause -> ...
6   | Implication (clause, lit) -> ...
```

Depending on the result of `unit_propagate`, we will either `backtrack_learn`:

```
sat/cdcl.ml
1 let rec bcp (level : int) (trail : Trail.trail) (formula : Formula.formula) : Solution.solution =
2   ...
3   and backtrack_learn ~level clause trail formula =
4     let trail' = Trail.backjump ~level trail in
5     let formula' = clause :: formula in
6     bcp level trail' formula'
```

or `decide`:

```
sat/cdcl.ml
1 and decide ~lit level trail =
2   let next_lvl = level + 1 in
3   let trail' = Trail.decided ~lit next_lvl trail in
4   bcp next_lvl trail'
```

So let’s take a look at what `unit_propagate` does.

Unit Propagation

Unit propagation finds literals that must be true.

For example:

$$p \wedge \neg q \wedge (\neg p \vee q \vee \neg r)$$

forces:

$$p = \text{true}$$

and:

$$q = \text{false}$$

because p and $\neg q$ are unit clauses.

The main `unit_propagate` result is one of three useful cases: **Decide**, **Conflict**, or **Implication**.

```
sat/cdcl.ml
1 let unit_propagate formula model =
2   ...
3   match Model.eval_clause clause model with
4   | `Falsified -> Conflict clause
5   | `Undecided [lit] -> Implication (clause, lit)
6   | _ -> search_unit clauses'
```

For example, if:

$$(\neg p \vee q \vee r)$$

and the model says $p = \text{true}$ and $q = \text{false}$, then unit propagation forces:

$$r = \text{true}$$

Deciding and Conflicts

Sometimes nothing is forced:

$$(p \vee q) \wedge (\neg p \vee r)$$

So CDCL makes a **decision**, meaning it guesses an unassigned variable:

If CDCL guesses:

```
sat/cdcl.ml
1 let rec bcp (level : int) (trail : Trail.trail) (formula : Formula.formula) : Solution.solution =
2   let model = Trail.to_model trail in
3   begin match unit_propagate formula model with
4   | Decide ->
5     let atoms = List.map Formula.atom_from_literal model in
6     begin match Formula.find_free_variable_opt atoms formula with
7     | None -> ...
8     | Some x -> decide ~lit:(Formula.pos x) level trail formula
9     ...
```

$$p = \text{true}$$

then:

$$(\neg p \vee r)$$

forces:

$$r = \text{true}$$

But guesses can lead to conflicts. For example:

$$(p \vee q) \wedge (\neg p \vee q) \wedge (p \vee \neg q) \wedge (\neg p \vee \neg q)$$

If CDCL guesses $p = \text{true}$, then it is forced to set $q = \text{true}$. But then $(\neg p \vee \neg q)$ becomes false.

CDCL learns from this conflict instead of blindly backtracking. In this case, it can learn:

$$\neg p \vee \neg q$$

which prevents the same bad assignment from being repeated.

A Full Example

Consider:

$$(p \vee q) \wedge (\neg p \vee r) \wedge (\neg r \vee q)$$

Suppose Blue3 makes the bad decision:

$$q = \text{false}$$

Then $(p \vee q)$ forces $p = \text{true}$, and $(\neg p \vee r)$ forces $r = \text{true}$.

Now:

$$(\neg r \vee q)$$

is false, so Blue3 gets a conflict.

sat/cdcl.ml


```

1 let rec bcp (level : int) (trail : Trail.trail) (formula : Formula.formula) : Solution.solution =
2   let model = Trail.to_model trail in
3   begin match unit_propagate formula model with
4     ...
5     | Conflict clause ->
6       let clause', backtrack_lvl = Trail.analyze_conflict ~clause level trail in

```

Here, the conflict clause is:

$$(\neg r \vee q)$$

Since r came from $(\neg p \vee r)$, resolving gives:

$$(q \vee \neg p)$$

Since p came from $(p \vee q)$, resolving again gives:

$$q$$

So Blue3 learns:

```

          q
let trail' = Trail.backjump ~level trail in
let formula' = clause :: formula in
bcp level trail' formula'

```

The formula effectively becomes:

$$q \wedge (p \vee q) \wedge (\neg p \vee r) \wedge (\neg r \vee q)$$

So Blue3 has learned that $q = \text{false}$ cannot work. After backtracking, $q = \text{true}$ is forced, and the formula becomes satisfiable.

The key idea is that a conflict may only mean the current branch is impossible. CDCL learns from the branch, backjumps, and continues.

SMT

Blue3 has two main pieces:

1. The **SAT solver**, which handles boolean structure.
2. The **IDL solver**, which handles difference constraints.

The SMT layer connects them:

SAT handles boolean structure; the theory solver checks whether the selected theory literals are consistent.

Theory atoms and results

Blue3 represents theory-level boolean expressions as `Theory.atoms`. Theory literals are signed atoms, either positive or negative. A theory solver receives a conjunction of these literals and returns one of:

- `Theory_sat`: the literals are consistent.
- `Theory_unsat`: the literals are inconsistent, with a core explaining why.
- `Theory_split`: the solver needs more case splits.
- `Theory_unknown`: Blue3 should fall back to the next solver.

```
type 'k theory_solution =  
  | Theory_unknown | Theory_sat of 'k Model.t  
  | Theory_unsat of 'k core  
  | Theory_split of 'k formula
```

Boolean abstraction

The SAT solver cannot directly reason about theory atoms, so Blue3 maps them to fresh SAT atoms.

For example:

```
p1 := (6 <= a)  
p2 := (a < 0)
```

So:

$$(6 \leq a) \wedge (a < 0)$$

becomes:

$$p_1 \wedge p_2$$

Blue3 keeps both directions of this mapping so it can move between SAT literals and theory literals.

```
Hashtbl.add conn.to_sat atom sat_atom;  
Hashtbl.add conn.from_sat sat_atom atom;  
sat_atom
```

The CDCL(T) loop

The main SMT loop is:

1. Abstract SMT into SAT.
2. Run CDCL.
3. Convert the SAT model back into theory literals.
4. Ask the theory solver whether they are consistent.

5. If theory says UNSAT, learn a clause and retry.
6. If theory says SPLIT, add split clauses and retry.

This is CDCL plus a theory solver: CDCL(T).

```
match Sat.Cdcl.cdcl sat_formula with
| UNSAT -> Solution.Unsat
| SAT model ->
  let theory_lits = make_theory_literals model conn in
  match solver theory_lits with ...
```

Example and Theory Learning

Consider:

$$(6 \leq a) \wedge (a < 0)$$

Blue3 abstracts this to:

```
p1 := (6 <= a)
p2 := (a < 0)
```

The SAT formula is:

$$p_1 \wedge p_2$$

SAT can satisfy this with both p_1 and p_2 set to true. But when Blue3 maps that model back to theory literals, IDL sees:

$$(6 \leq a) \wedge (a < 0)$$

which normalizes to:

$$0 - a \leq -6$$

and:

$$a - 0 \leq -1$$

These encode as graph edges:

$$(a, 0, -6)$$

and:

$$(0, a, -1)$$

Bellman-Ford finds a negative cycle, so the theory solver returns `Theory_unsat` with this core:

$$(6 \leq a) \wedge (a < 0)$$

Since the core was $p_1 \wedge p_2$, Blue3 learns:

$$\neg p_1 \vee \neg p_2$$

```
core
|> List.map (fun lit -> abstract_literal lit conn)
|> List.map Sat.Formula.negate
```

The SAT formula becomes:

$$p_1 \wedge p_2 \wedge (\neg p_1 \vee \neg p_2)$$

Now CDCL can prove the boolean abstraction itself is impossible.

Theory Splitting

Disequality needs a split:

$$x \neq y$$

means:

$$x \leq y - 1$$

or:

$$y + 1 \leq x$$

Blue3 adds this as a guarded split clause:

$$(x = y) \vee (x \leq y - 1) \vee (y + 1 \leq x)$$

So the theory solver gives SAT more structure, and the loop continues.

```
| Theory_split clauses ->
  let sat_formula' = abstract_clauses clauses conn sat_formula in
  loop conn sat_formula'
```

The final Blue3 wrapper

Blue3 falls back if the formula contains operations outside its internal fragment, like multiplication, division, modulus, or general addition.

```
let contains_unsolvable formula =  
  Formula.contains_binops [Times; Divide; Modulus; Plus] formula
```

So the final architecture is:

```
Formula -> Blue3 checks -> CDCL(T)  
CDCL(T) -> SAT abstraction -> CDCL -> Theory solver  
Theory solver -> learning / splitting  
Result -> SAT / UNSAT / fallback
```

Benchmarks and Last Words

These measurements use the Benchmark library on 179 formulas from a real `ceval` run. Each formula was solved 2000 times by each solver, and the averages were saved.

Measurements

Average runtimes

blue3	z3
241.0 μ s	359.0 μ s

On average, blue3 outperforms z3 by just under ~40%.

Blue3-only count

count
89

Out of the 179 formulas solved by Blue3, it solved 89 of them without having to fallback to Z3.

Z3-deferred count

count
90

Out of the 179 formulas solved by Blue3, it deferred 90 formulas to Z3 .

Deferred overhead

cases	diff	pct
90	-12.83 μ s	-2.41%

On average, running Z3 through the Blue3 solve pipeline incurs a ~13 microsecond overhead over running Z3 alone.

Fast cases

cases	diff	pct
146	146.61 μ s	61.54%

On the cases that Blue3 was able to solve faster than Z3, regardless of if it deferred to Z3 or not, it was able to solve it was around ~60% faster. While a more thorough analysis would need to be done to say this for certain, I am guessing that the 146 – 89 cases that were deferred to Z3 were faster than the Z3 call alone because of the simplifications Blue3 performs.

Slow cases

cases	diff	pct
33	10.05 μ s	3.2%

When Z3-only solver outperformed Blue3, it was by 3.2%.

Top 5 fastest Blue3-only cases

id	formula	blue3	z3	diff
8	$(0 < a) \wedge ((a + 1) \leq 1)$	0.21 μ s	91.41 μ s	-91.2 μ s
9	$(0 < a) \wedge ((a + 1) \leq a)$	0.28 μ s	88.46 μ s	-88.18 μ s
56	$(\text{not } (a = 0)) \wedge ((a + 10) = 0)$	0.29 μ s	230.75 μ s	-230.46 μ s
117	$(0 \leq a) \wedge ((1 + a) < 0)$	0.3 μ s	124.54 μ s	-124.24 μ s
178	$(0 \leq a) \wedge ((a + 2) < 0)$	0.31 μ s	127.94 μ s	-127.64 μ s

Blue3 greatly outperforms Z3 on small IDL cases.

Top 5 slowest Blue3-only cases

id	formula	blue3	z3	diff
167	(not (a = 108)) ^ (not (a = 105)) ^ (not (a = 98)) ^ (not (a = 97)) ^ (not (a = 61)) ^ (not (a = 45)) ^ (not (a = 43)) ^ (not (a = 42)) ^ (not (a = 41)) ^ (not (a = 40)) ^ (not (a = 32)) ^ (48 <= a)	177.42 μ s	477.72 μ s	- 300.29 μ s
168	(48 <= a) ^ (not (a = 108)) ^ (not (a = 105)) ^ (not (a = 98)) ^ (not (a = 97)) ^ (not (a = 61)) ^ (not (a = 45)) ^ (not (a = 43)) ^ (not (a = 42)) ^ (not (a = 41)) ^ (not (a = 40)) ^ (not (a = 32)) ^ (57 < a)	175.71 μ s	456.12 μ s	- 280.41 μ s
172	(65 <= a) ^ (48 <= a) ^ (57 < a) ^ (not (a = 108)) ^ (not (a = 105)) ^ (not (a = 98)) ^ (not (a = 97)) ^ (not (a = 61)) ^ (not (a = 45)) ^ (not (a = 43)) ^ (not (a = 42)) ^ (not (a = 41)) ^ (not (a = 40)) ^ (not (a = 32)) ^ (90 < a)	103.67 μ s	507.73 μ s	- 404.05 μ s
170	(48 <= a) ^ (57 < a) ^ (not (a = 108)) ^ (not (a = 105)) ^ (not (a = 98)) ^ (not (a = 97)) ^ (not (a = 61)) ^ (not (a = 45)) ^ (not (a = 43)) ^ (not (a = 42)) ^ (not (a = 41)) ^ (not (a = 40)) ^ (not (a = 32)) ^ (65 <= a)	101.82 μ s	477.36 μ s	- 375.55 μ s
175	(65 <= a) ^ (48 <= a) ^ (90 < a) ^ (57 < a) ^ (not (a = 108)) ^ (not (a = 105)) ^ (not (a = 98)) ^ (not (a = 97)) ^ (not (a = 61)) ^ (not (a = 45)) ^ (not (a = 43)) ^ (not (a = 42)) ^ (not (a = 41)) ^ (not (a = 40)) ^ (not (a = 32)) ^ (97 <= a)	66.57 μ s	492.07 μ s	- 425.5 μ s

These are Blue3's slowest solved cases that are purely IDL-compatible. They contain many disequalities and redundant lower bounds, causing more simplification and case-analysis work before the formula collapses and outperforms calling Z3 alone.

Cases Z3 beat Blue3

id	formula	blue3	z3
94	$(b \leq a) \wedge (0 < b) \wedge (0 < a) \wedge (\text{not } ((a \% b) = 0)) \wedge (\text{not } (b = 0)) \wedge ((b + 1) \leq c)$	1163.14	1157.67
105	$(0 < a) \wedge (0 < b) \wedge (0 < c) \wedge (\text{not } ((c \% b) = 0)) \wedge (\text{not } (b = 0)) \wedge (b < a)$	1065.32	1047.21
103	$(0 < a) \wedge (0 < b) \wedge (0 < c) \wedge (\text{not } ((c \% b) = 0)) \wedge (\text{not } (b = 0)) \wedge (a \leq b)$	996.61	985.42
70	$(\text{not } ((a \% 2) = 0)) \wedge (\text{not } (a = 0)) \wedge (\text{not } (((a - 1) \% 2) = 0))$	703.25	661.79
102	$((b \% a) = 0) \wedge (0 < a) \wedge (0 < b) \wedge (\text{not } (a = 0)) \wedge (((b * a) / a) < b)$	664.39	655.5

Z3 wins on formulas involving modulus, multiplication, and division, which Blue3 intentionally leaves to a general-purpose solver, because the simplifications Blue3 performs on these formulas before sending to Z3 does not make up for the overhead of those simplifications.

Last words

$P = NP$ fascinates me because it gets at the core of problem solving. If checking a solution and finding a solution are secretly the same kind of task, then creativity, insight, and cleverness start to look very different.

Theoretical computer scientist Scott Aaronson once said:

“If $P=NP$, then the world would be a profoundly different place than we usually assume it to be. There would be no special value in ‘creative leaps,’ no fundamental gap between solving a problem and recognizing the solution once it’s found. Everyone who could appreciate a symphony would be Mozart; everyone who could follow a step-by-step argument would be Gauss; everyone who could recognize a good investment strategy would be Warren Buffett.”

For however long we do not know whether $P = NP$, there is still room to believe that something about human problem solving is special and to believe there is something irreducible about how we notice, create, and care about things.

Or maybe not. But until $P = NP$ is proven either way, we can’t say for sure.

And that’s pretty poetic, because wanting to be special, even if just a little, is one of the most human things there is.

Blue3 obviously did not answer $P = NP$, but building it showed me why these questions matter, and I hope it did for you as well.